

Rendering Lists and Array Methods

Rendering Lists & JavaScript Array Methods in React

Today You Will Learn

- Rendering Lists using `map()`
- `filter()`
- `find()`
- `sort()`
- `reduce()`
- Conditional Rendering
- Building Dynamic UI using Arrays

📌 **Learning Outcome:** By the end of today's session, you will be able to display, search, filter, sort, and summarize data in React applications using JavaScript array methods.



Rendering Lists using map()

What is map()?

The map() method is used to create a new array by transforming each element.

Example Data

```
const employees = [  
  { id: 1, name: "Rahul" },  
  { id: 2, name: "Priya" },  
  { id: 3, name: "John" }  
];
```

Rendering in React

```
{employees.map(employee => (  
  <h3>{employee.name}</h3>  
))}
```

Real World

Product Cards

Employee Cards

Student Lists

Notifications

Orders

filter()

What is filter()?

Returns only the elements that satisfy a condition.

Example

```
const products = [  
  { id:1, name:"Laptop", price:60000 },  
  { id:2, name:"Mouse", price:800 },  
  { id:3, name:"Phone", price:35000 }  
];
```

Filter expensive products

```
const expensiveProducts = products.filter(  
  product => product.price > 10000  
);
```

Render

```
expensiveProducts.map(product => (  
  <h3>{product.name}</h3>  
))
```

Real World

- Search Results
- Product Categories
- Active Users
- Completed Tasks

find()

What is find()?

Returns the **first matching object**.

Example

```
const student = students.find(  
  student => student.id === 2  
);
```

Output

```
{ id:2, name:"Priya" }
```

Display

```
<h2>{student.name}</h2>
```

Real World



Find User



Find Product



Find Employee



Find Order

sort()

What is sort()?

Sorts array elements.

Ascending Price

```
const sortedProducts = [...products].sort(  
  (a,b) => a.price - b.price  
);
```

Descending Price

```
const sortedProducts = [...products].sort(  
  (a,b) => b.price - a.price  
);
```

Sort Names

```
products.sort(  
  (a,b) => a.name.localeCompare(b.name)  
);
```

Real World

Lowest Price

Highest Salary

Alphabetical List

Ranking Systems

reduce()

What is reduce()?

Combines all values into a single result.

Example

```
const cart = [  
  { price:200 },  
  { price:400 },  
  { price:600 }  
];
```

Calculate Total

```
const total = cart.reduce(  
  (sum, item) => sum + item.price, 0  
);
```

Display

```
<h2>Total : ₹{total}</h2>
```

Real World

Shopping Cart Total

Employee Salary Total

Attendance Count

Revenue Calculation

Conditional Rendering

Show UI based on conditions

Ternary Operator

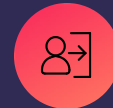
```
const loggedIn = true;

{ loggedIn ?
  <h2>Welcome</h2>
  :
  <h2>Please Login</h2>
}
```

AND Operator

```
{ cart.length > 0 &&
  <h2>Items Available</h2>
}
```

Real World



Login



Loading Spinner



Empty Cart



Error Messages

Array Methods Together

Complete Example

```
const products = [  
  { id:1, name:"Laptop", price:60000 },  
  { id:2, name:"Mouse", price:800 },  
  { id:3, name:"Phone", price:35000 }  
];
```

Filter and Display

```
products  
  .filter(product => product.price > 10000)  
  .sort((a,b) => a.price - b.price)  
  .map(product => (  
    <h3>{product.name}</h3>  
    <p>₹ {product.price}</p>  
  ))
```

Concepts Used

✓ filter()

✓ sort()

✓ map()

Mini Project: Employee Dashboard

Objective

Create an Employee Dashboard using Bootstrap Cards.

Employee Object

```
{  
  id:1,  
  name:"Rahul",  
  department:"Development",  
  salary:60000,  
  experience:3  
}
```

Features

- Display all employees using map()
- Filter salary above ₹50,000
- Find employee by ID
- Sort employees by salary
- Calculate total salary using reduce()

Bootstrap Card

Each card should display:

Employee Name

Department

Salary

Experience

View Profile Button

Concepts Used

MAP()

FILTER()

SORT()

REDUCE()

FIND()

BOOTSTRAP CARDS

Homework Assignment

Homework 1: Student Result Dashboard

Create a dashboard using Bootstrap Cards.

Student Details

- Photo
- Name
- Roll Number
- Marks
- Grade
- Status

Features

1. **Display Students** — Use `map()`
2. **Display Passed Students** — Use `filter()`
3. **Find Student** — Search using Roll Number. Use `find()`
4. **Sort Students** — Highest Marks First. Use `sort()`
5. **Calculate** — Total Marks, Average Marks. Use `reduce()`
6. **Conditional Rendering** — If Marks ≥ 35  PASS, Else  FAIL

Submission Checklist

-  `map()`
-  `filter()`
-  `find()`
-  `sort()`
-  `reduce()`
-  Conditional Rendering
-  Bootstrap Cards
-  Responsive Layout

Expected Outcome

Students will be able to build dynamic React applications by rendering lists, filtering data, searching records, sorting collections, calculating totals, and displaying different UI based on conditions. These array methods are used extensively in production React applications such as e-commerce websites, dashboards, HR management systems, and admin panels.